

Data Structures Project: Array-Based Linked List Management

Part 1: Node Management System (Header File)

Develop a C++ header file, tentatively named **nodePool.h**, to manage a fixed-size storage pool for an array-based linked list. This system is the backbone for dynamic allocation within a static array.

Requirements for nodePool.h:

1. **Node Structure Definition:** Define a structure (or class) for the nodes. Each node must internally manage two key pieces of information:
 - The **data** element (The type of data, e.g., **ElementType**, should be defined clearly by the student).
 - A **link/pointer** to the successor node within the array.
 - The node structure (or class) should include appropriate constructors (at least one no-argument and one argument-receiving constructor) to manage the initial state of the link field. The student must decide on an appropriate non-existent location value (the **NULL** equivalent).
2. **Storage Pool Management:** Within the header file (or an associated implementation if you prefer a separate file for functions), define the array that serves as the **fixed storage pool** for all nodes, and define a mechanism to track the **first available (free)** node. The student should select a suitable, reasonable size for this array (N=10).
3. **Core Pool Functions:** Implement the following functions/methods to manage the free nodes:
 - **Initialization Routine:** A function to set up the entire array of nodes initially so that every node is linked together to form a **Free List**.
 - **Allocation Function (Acquire Node):** A function that simulates the memory allocation process by retrieving the index of the next available node from the Free List. This function must handle the removal of the returned node from the available pool.
 - **Deallocation Function (Release Node):** A function that returns a previously used node's index back to the storage pool, making it available for future use. The most efficient way to achieve this (inserting the node back into the free list) should be implemented.

Part 2: The Array-Based List Class

Define a C++ class, tentatively named **ArrayBasedList**, which implements the functionality of a linked list entirely using the array allocation system developed in Part 1. This class must handle list operations using only the integer indices (pointers) managed by the **nodePool.h** system.

Requirements for ArrayBasedList:

1. **Private Data:** The class must maintain a single pointer (an integer index) to the **first element** of the active list. It must also use (include or instantiate) the node management system defined in Part 1.
2. **Standard List Operations:** Implement the following core methods, ensuring all operations use the allocation and deallocation functions from Part 1 when necessary:
 - **Constructors** and a **Destructor** (The destructor must return all currently used nodes back to the Free List for proper cleanup).
 - **Assignment Operator Overloading (operator=):** A robust implementation for deep copying the list.
 - **State Check:** A method to determine if the list is **empty**.
 - **Insertion:** A method (insert) that places a new data value at a specified index/position within the active list. This requires acquiring a new node from the pool.
 - **Deletion:** A method (remove or delete) that logically removes a node at a specified index/position, returning the node's index back to the Free List.
 - **Searching:** A method (search) to find a given data value and return its position (index) or an indication that it was not found.
 - **Traversal:** A method (display or print) that traverses the active list and outputs the data elements in order.

Part 3: The User-Friendly Tester (tester.cpp)

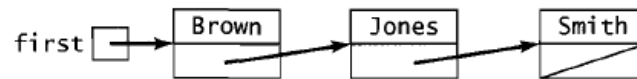
Create a driver program that thoroughly tests the **ArrayBasedList** class. The tester should demonstrate a **robust and interactive user interface** that clearly shows the state of the list after each operation (insertion, deletion, etc.).

- The tester should include a menu allowing the user to choose list operations.
- The output should visually represent the list's contents *and* should also include a way to show that the free list is being managed correctly (e.g., a count of available nodes, or a simple display of the free pointer's value).

Deliverables

Submit the following files: **nodePool.h**, **ArrayBasedList.h**, and **tester.cpp**, along with a complete documentation file similar to the standards discussed in class (File, Function, and Code levels).

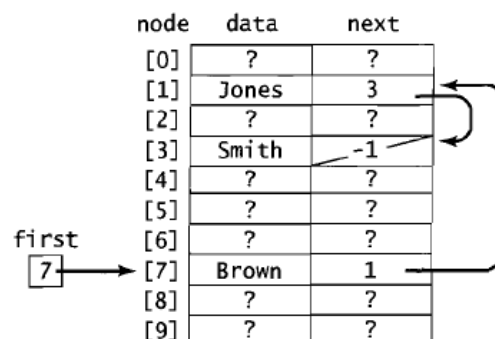
To illustrate, consider a simple linked list containing the names Brown, Jones, and Smith in this order:



Here **first** is a variable of type int and it points to the first node by storing its location in the storage pool node. Suppose for simplicity that NUM_NODES is 10, so that the array node constructed by the preceding declarations consists of 10 structs, each of which has a data member for storing a name and a next member for storing the location of its successor. The nodes of the linked list can be stored in any three of these array locations, provided that the links are appropriately set and first is maintained as a pointer to the first node.

For example, the first node might be stored in location 7 in this array, the second node in location 1, and the third in location 3. Thus, first would have the value 7, node[7].data would store the string "Brown", and node[7].next would have the value 1. Similarly, we would have node[1].data = "Jones" and node[1].next = 3. The last node would be stored in location 3 so that node[3].data would have the value "Smith".

The following diagram displays the contents of the array node and indicates how the next members connect these nodes. The question marks in some array locations indicate undetermined values because these nodes are not used to store this linked list.



To traverse this list, displaying the names in order, we begin by finding the location of the first node by using the pointer first. Since first has the value 7, the first name displayed is "Brown", stored in node[7].data.

Following the next member leads us to array location node[7].next = 1, where the name "Jones" is stored in node [1].data, and then to location node[1].next = 3, where "Smith" is stored.

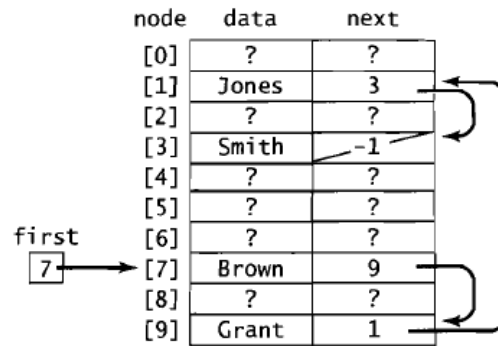
The null value -1 for node[3].next signals that this is the last node in the list.

In general, to traverse any linked list, we use the method given in the traversal algorithm:

1. Initialize ptr to first.
 2. While (ptr != NULL_VALUE) do the following:
 - a. Process the data part of the node pointed to by ptr.
 - b. Set ptr equal to the next part of the node pointed to by ptr.
- End while

Now suppose we wish to insert a new name into this list, for example, to insert "Grant" after "Brown". We must first obtain a new node in which to store this name.

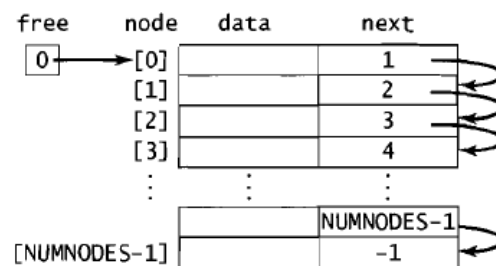
Seven locations are available, namely, positions 0, 2, 4, 5, 6, 8, and 9 in the array. Let us assume for now that this storage pool of available nodes has been organized in such a way that a call to a function **newNode()** returns the index 9 as the location of an available node. The new name is then inserted into the list using the method described in the preceding section: node[9].data is set equal to "Grant"; node[9].next is set equal to 2 so that it points to the successor of "Brown"; and the link field node[7].next of the predecessor is set equal to 9.



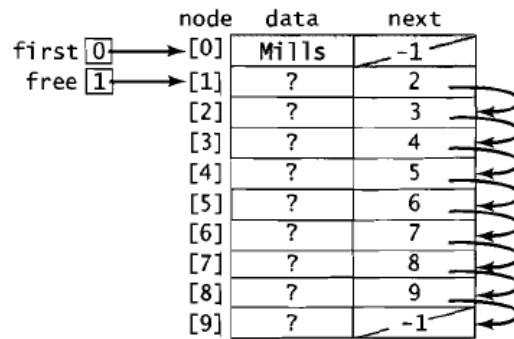
Organizing the Storage Pool

This example illustrates that the elements of the array node are of two kinds. Some of the array locations—namely 1,3,7, and 9—are used to store nodes of the linked list. The others represent unused "free" nodes that are available for storing new items as they are inserted into the list. We have described in detail how the nodes used to store list elements are organized, and we must now consider how to structure the storage pool of available nodes.

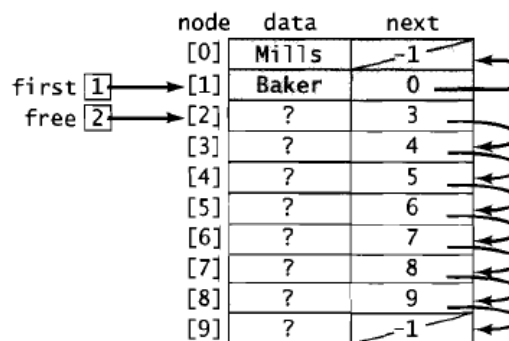
One simple way is to begin by linking all the nodes together to form the storage pool by letting the first node point to the second, the second to the third, and so on. The link field of the last node will be null, and a pointer free is set equal to 0 to provide access to the first node in this storage pool:



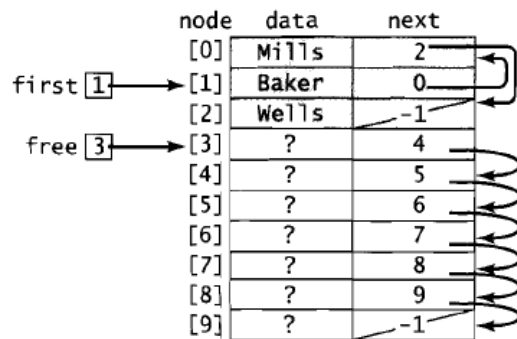
A function call `ptr = newNode()` returns the location of a free node by assigning to `ptr` the value `free` and removing that node from the free list by setting `free` equal to `node[free].next`. Thus, if "Mills" is the first name to be inserted into a linked list, it will be stored in the first position of the array node, because `free` has the value 0; `first` will be set to 0; and `free` will become 1:



If "Baker" is the next name to be inserted, it will be stored in location 1 because this is the value of free, and free will be set equal to 2. If the list is to be maintained in alphabetical order, first will be set equal to 1; node[1].next will be set equal to 0; and node [0].next will be set equal to -1:

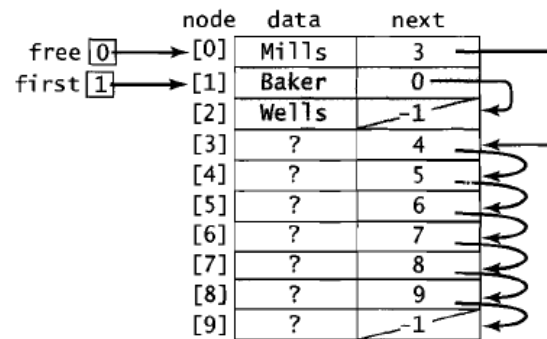


If "Wells" is the next name inserted into the list, the following configuration will result:



When a node is deleted from a linked list, it should be returned to the storage pool of free nodes so that it can be reused later to store some other list element. A function call **deleteNode(ptr)** simply inserts the node pointed to by ptr at the beginning of the free list by first setting node[ptr].next equal to free and then setting free equal to ptr.

For example, deleting the name "Mi11s" from the preceding linked list produces the following configuration:



Note that it is not necessary to actually remove the string "Mills" from the data part of this node because changing the link of its predecessor has *logically* removed it from the linked list. This string "Mills" will be overwritten when this node is used to store a new name.